

Data Anomaly

Data Anomalies are inconsistencies or errors in the data that arise from poor database design, such as storing redundant information in a single, non-normalized table. These flaws compromise data integrity and can lead to unreliable operations and flawed decision-making.

Data anomalies usually appear in three common forms. An update anomaly happens when a single fact is repeated across many rows, so changing it requires multiple edits; if you update some rows and miss others, the database now contains conflicting versions of the truth.

An insertion anomaly happens when the table structure forces you to insert unrelated data just to record a new fact; you might want to record a new course, but the table demands a student row first. A deletion anomaly happens when removing a row unintentionally removes a fact you still need; if the last student enrolled in a course is deleted, the only stored information about that course may disappear too.

ENROLLMENT_ALL

StudentID	StudentName	DeptName	CourseID	CourseTitle	InstructorName	Grade
S001	Ada James	Computer Science	C101	Database Systems	Dr. Bello	A
S002	Tunde Ola	Computer Science	C101	Database Systems	Dr. Bello	B
S003	Zainab Ali	Statistics	C205	Data Mining	Dr. Okafor	A
S001	Ada James	Computer Science	C205	Data Mining	Dr. Okafor	B

Update anomaly

If Dr. Bello changes their name, it must be updated everywhere C101 appears, or the table becomes inconsistent.

Insertion anomaly

If a new course exists but no one has enrolled yet, it cannot be stored cleanly in this table.

Deletion anomaly

If the last row for C205 is deleted, the database may lose the only record of that course title and instructor.

Database Normalization

Database normalization is the process of organizing a database so that each table stores facts about a single subject in a clean, logical way. The main goal is to reduce redundancy and prevent the anomalies discussed earlier. Normalization does this by separating mixed information into related tables and ensuring that each non-key attribute depends on the key in a proper way.

A major concept that supports normalization is functional dependency. A functional dependency exists in a relation when the primary key uniquely determines a non-key attribute. It is written as $K \rightarrow A$, where K is a key (the determinant) and A is a non-key attribute. This means that each key value maps to exactly one value of the non-key attribute. For example, $\text{StudentID} \rightarrow \text{StudentName, Email, Address}$ means a given StudentID always identifies exactly one StudentName, Email, Address. Functional dependency helps us decide which attributes belong together in the same table and which should be moved into separate tables to avoid redundancy.

Normalization is usually described in stages called normal forms. In First Normal Form (1NF), the focus is on making sure each column holds atomic values, meaning each cell contains a single value, not a list or repeating group. A relation is in First Normal Form (1NF) when every attribute contains atomic (single, indivisible) values, and there are no repeating groups or lists stored in a single column.

Put simply, a cell should not hold multiple values, and the table should look like a proper grid where each field is one value per row. When you store multiple values in one cell (like many phone numbers or many courses), searching, updating, and joining become messy and inconsistent. Consider the relation below.

Student Relation

Staff_Number	Name	Unit_Num	Unit_Name
N0001	Chukwudi Emeka	35	Accounting
N0002	Divine Miracle	15	Marketing
N0003	Adewara Joseph	35	Accounting
N0004	Arnold Wisdom	35	Accounting

Student Relation in 1NF - A relation is in First Normal Form (1NF) when every attribute contains atomic (single, indivisible) values, and there are no repeating groups or lists stored in a single column.

Staff_Number	FirstName	LastName	Unit_Num	Unit_Name
N0001	Chukwudi	Emeka	35	Accounting
N0002	Divine	Miracle	15	Marketing
N0003	Adewara	Joseph	35	Accounting
N0004	Arnold	Wisdom	35	Accounting

In Second Normal Form (2NF), the focus is on removing partial dependency that happens when a table has a composite key and some non-key attributes depend on only part of that key. A relation is in Second Normal Form (2NF) when:

1. It is already in 1NF, and
2. Every non-primary key attribute is fully functionally dependent upon primary key attributes.

This mainly applies when a table has a composite (multi-attribute) key. 2NF is violated when a non-key attribute depends on only part of that composite key. That situation is called a partial dependency.

Student Relation in 2NF

A relation is in Second Normal Form (2NF) when:

1. It is already in 1NF, and

Student Relation in 1NF - A relation is in First Normal Form (1NF) when every attribute contains atomic (single, indivisible) values, and there are no repeating groups or lists stored in a single column.

Staff_Number	FirstName	LastName	Unit_Num	Unit_Name
N0001	Chukwudi	Emeka	35	Accounting
N0002	Divine	Miracle	15	Marketing
N0003	Adewara	Joseph	35	Accounting
N0004	Arnold	Wisdom	35	Accounting

2. Every non-primary key attribute is fully functionally dependent upon primary key attributes.

Student relation 2NF

Staff_Number	Last_Name	First_Name
N0001	Chukwudi	Emeka
N0002	Divine	Miracle
N0003	Adewara	Joseph
N0004	Arnold	Wisdom

Department relation 2NF

Unit_Num	Unit_Name
35	Accounting
15	Marketing